CAPSTONE PROJECT REPORT

AIR POLLUTION AI: CNN IMAGE CLASSIFICATION + LIVE AQI INTEGRATION

Submitted by: _____

Course / Batch: _____

Institute: _____

Submission deadline: 03 November 2026

Student Declaration

I declare that this capstone report describes the project implementation and sources used. I will include only results generated by executing the code and will acknowledge third-party datasets and APIs.

1. Abstract

This project combines two independent components: (a) a Convolutional Neural Network (CNN) that classifies an uploaded outdoor photograph into one of six visual air-pollution classes, and (b) a live air-quality data integration module that retrieves model-based pollutant estimates for a user-selected city. The CNN predicts a visual class and confidence score. The Open-Meteo Air Quality API supplies model-based US AQI and pollutant concentrations such as PM2.5, PM10, CO, NO2, SO2 and O3. The dashboard keeps these outputs separate because a photograph cannot directly measure pollutant concentrations.

2. Introduction

Air pollution is a major environmental and public-health issue. Citizens benefit from understandable air-quality information. Image classification can recognize visual patterns in outdoor scenes, while atmospheric-composition models and monitoring stations provide numerical pollutant data. This capstone demonstrates how these two types of information can be presented together in a beginner-friendly Python application.

3. Problem Statement

To create a simple AI dashboard where a user uploads an outdoor image to receive a visual pollution-class prediction and selects a city to retrieve current model-based air-quality estimates and pollutant concentrations.

4. Objectives

• Build a six-class CNN image classifier.\n• Predict a visual class and confidence score from an uploaded image.\n• Integrate a city search and air-quality API.\n• Display US AQI and PM2.5, PM10, CO, NO2, SO2 and O3 estimates.\n• Explain data sources, limitations and responsible interpretation.

5. System Architecture

Input A: uploaded image → resize to 128 × 128 → normalize pixel values → trained CNN → six-class visual prediction + confidence. Input B: city name → Open-Meteo Geocoding API → latitude/longitude → Open-Meteo Air Quality API → model-based US AQI and pollutant concentrations → dashboard. The two outputs are displayed separately and are not mathematically combined.

6. Dataset

The image model uses the Air Pollution Image Dataset from India and Nepal. The dataset repository describes 12,240 images at 224 × 224 resolution and six classes: Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy, Very Unhealthy and Hazardous. The raw archive is distributed by Kaggle and must be downloaded separately. The user must follow the dataset repository's license and attribution instructions.

7. CNN Classes and Meaning

The CNN predicts the dataset's six visual labels. These labels are not the same as India's CPCB National AQI categories. A CNN confidence score (for example, 87%) means the model assigns that score to its predicted image class; it is not the probability that the measured air quality is truly in that band.

8. Live AQI Integration

The app uses Open-Meteo Geocoding to convert a city name to coordinates. It then calls the Open-Meteo Air Quality API for current US AQI and pollutant concentrations. It also requests recent hourly PM2.5 and PM10 values for a small trend chart. Open-Meteo documents CAMS-based atmospheric-composition forecast data; the spatial resolution varies by data domain. These values are model estimates and should not be described as direct readings from a nearby CPCB monitoring station.

9. Pollutants Displayed

PM2.5: fine particulate matter, µg/m³. PM10: inhalable particulate matter, µg/m³. CO: carbon monoxide, µg/m³ in this API response. NO2: nitrogen dioxide, µg/m³. SO2: sulphur dioxide, µg/m³. O3: ozone, µg/m³. US AQI is a separate index based on United States EPA breakpoints and is not India's CPCB National AQI.

10. Tools and Technologies

Python 3.10/3.11, TensorFlow/Keras, Streamlit, Requests, Pillow, NumPy, Pandas, Matplotlib and scikit-learn. The application runs locally in a browser using Streamlit.

11. CNN Methodology

The dataset is split into 70% training, 15% validation and 15% test images using stratified sampling. Images are resized to 128 × 128 and normalized to 0–1. Small augmentation is applied to training images. The CNN contains three convolution/max-pooling

blocks (32, 64 and 128 filters), a flatten layer, a dense layer with 128 units, dropout and a six-neuron softmax output. Adam optimizer and categorical cross-entropy are used. Early stopping monitors validation loss.

12. Live Data Workflow

The user enters a city and uploads an outdoor photo. The app loads the saved CNN model and predicts the visual class. It geocodes the city, requests current pollutant estimates and US AQI, then shows pollutant values and recent hourly PM trends. The response timestamp and timezone are shown where supplied by the API.

13. Evaluation

CNN evaluation uses held-out test accuracy, precision, recall, F1-score and a confusion matrix. The training script saves training curves and a classification report. Live-data integration can be tested by checking whether city lookup succeeds, whether the API returns values and timestamps, and whether missing values are handled without crashing. Do not invent accuracy or pollutant values in the report.

14. Results

After running train.py, copy the measured test accuracy from outputs/experiment_summary.txt into this section: _____. Insert outputs/training_curves.png and outputs/confusion_matrix.png. After running app.py, capture a screenshot showing the uploaded image, CNN prediction, selected city, API timestamp and pollutant metrics. The model has not been pre-trained as part of this project package; actual results must be generated by the student.

15. Social Relevance

The prototype can support environmental awareness and demonstrate how AI and public data services can be combined. It can be extended for school projects, smart-city demonstrations and environmental education. It does not make regulatory or public-health decisions.

16. Limitations and Responsible Use

A photograph cannot measure PM2.5, PM10 or gaseous pollutant concentrations. Visual appearance is affected by fog, humidity, lighting, weather, camera settings and distance. The CNN may learn background or location-specific patterns. Open-Meteo data are model-based estimates on a grid, not guaranteed to represent street-level conditions. The displayed US AQI is not the Indian CPCB AQI. For official Indian readings, consult CPCB and nearby validated monitoring stations. Do not use this prototype for medical, emergency or official decisions.

17. Future Scope

Future work may include integrating a verified CPCB station feed where access and terms permit, adding low-cost calibrated sensors, training with more geographically diverse images, testing cross-city generalization, adding uncertainty estimates, and building a mobile-friendly interface.

18. Conclusion

This project demonstrates a practical beginner-level AI application combining CNN-based visual classification with a separate air-quality data API. It reports image predictions and pollutant estimates as distinct information sources, helping users understand both the capabilities and limits of image-based AI.

19. References

1. ICCC-Platform, Air Pollution Image Dataset from India and Nepal: https://github.com/ICCC-Platform/Air-Pollution-Image-Dataset-From-India-and-Nepal\n2. Kaggle dataset: https://www.kaggle.com/datasets/adarshrouniyar/air-pollution-image-dataset-from-india-and-nepal\n3. Open-Meteo Air Quality API documentation: https://open-meteo.com/en/docs/air-quality-api\n4. Open-Meteo Geocoding API documentation: https://open-meteo.com/en/docs/geocoding-api\n5. CPCB, About National Air Quality Index: https://app.cpcbccr.com/ccr_docs/About_AQI.pdf\n6. CPCB, National AQI Final Report: https://app.cpcbccr.com/ccr_docs/FINAL-REPORT_AQI_.pdf

Appendix A — Installation and Execution

Install Python 3.10/3.11. Download and extract the dataset using dataset/DATASET_INFO.txt. Install dependencies with pip install -r requirements.txt. Check DATASET_DIR in train.py. Run python train.py, then streamlit run app.py. Upload a photo, enter a city and analyse.